

Table of Contents

I. INTRODUCTION	2
II. C PROGRAM STRUCTURE AND EXAMPLES.....	2
A. Basic Hello World Program	2
B. Program with User Input.....	3
C. Program with Formatted Output	4
III. GCC COMPILATION PROCESS	6
A. Installing GCC on Linux.....	6
B. Four Stages of Compilation	6
C. COMPILATION STAGE EXAMPLES	7
IV. RETURN VALUES AND EXIT STATUS	9
A. Understanding Return Values	9
V. KNOWLEDGE CHECK QUESTIONS.....	10
A. Question 1.....	10
B. Question 2.....	10
C. Question 3	11
D. Question 4.....	11
E. Question 5	12
F. Question 6	12
VI. LAB TASKS COMPLETED	13
VII. CONCLUSION.....	14
VIII. GITHUB REPOSITORY	14

Lab 2: C Programming Basics Lab Report and Knowledge Check

I. INTRODUCTION

This lab report presents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The main focus of the lab was to understand the basics of C programming, GCC compilation, and commonly used Linux commands. The practical tasks also helped demonstrate how C source code is converted into an executable program through different compilation stages.

II. C PROGRAM STRUCTURE AND EXAMPLES

A. Basic Hello World Program

This program demonstrates the basic structure of a C program using the `stdio.h` header file and the `main()` function. It uses `printf()` to display a message and returns 0 to indicate successful execution.

```
aayushdas@Ubuntu:~$ mkdir ~/lab2
aayushdas@Ubuntu:~$ cd ~/lab2
aayushdas@Ubuntu:~/lab2$ pwd
/home/aayushdas/lab2
aayushdas@Ubuntu:~/lab2$ ls
```

```
aayushdas@Ubuntu:~/lab2$ nano hello.c
aayushdas@Ubuntu:~/lab2$ ls
hello.c
aayushdas@Ubuntu:~/lab2$ cat hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

The program was successfully compiled using GCC and executed in the Linux terminal. The output displayed “Hello, World!”, and the exit status was 0, indicating that the program completed successfully.

```
aayushdas@Ubuntu:~/lab2$ gcc hello.c -o hello
aayushdas@Ubuntu:~/lab2$ ls
hello  hello.c
aayushdas@Ubuntu:~/lab2$ ./hello
Hello, World!
aayushdas@Ubuntu:~/lab2$ echo $?
0
```

B. Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

```
aayushdas@Ubuntu:~/lab2$ nano input.c
aayushdas@Ubuntu:~/lab2$ cat input.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}
```

```
aayushdas@Ubuntu:~/lab2$ gcc input.c -o input
aayushdas@Ubuntu:~/lab2$ ./input
Enter a number: 25
You entered: 25
aayushdas@Ubuntu:~/lab2$ echo $?
0
```

C. Program with Formatted Output

Demonstrating multiple format specifiers and variables:

This program demonstrates how different types of data can be accepted from the user and displayed using formatted output. It uses a character array for the name, an integer for age, and a floating-point variable for height.

```
aayushdas@Ubuntu:~/lab2$ nano formatted.c
aayushdas@Ubuntu:~/lab2$ cat formatted.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

The program was compiled successfully using GCC and accepted the user's name, age, and height. The formatted output displayed each value using the appropriate format specifier, and the program returned an exit status of 0.

```
aayushdas@Ubuntu:~/lab2$ gcc formatted.c -o formatted
aayushdas@Ubuntu:~/lab2$ ./formatted
Enter your name: Aayush
Enter your age: 21
Enter your height (m): 1.78

--- User Info ---
Name: Aayush
Age: 21 years
Height: 1.78 m
aayushdas@Ubuntu:~/lab2$ echo $?
0
```

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

GCC is a compiler used to compile C programs on Linux. In this task, the GCC package was installed and its version was checked to confirm that the compiler was available and working correctly.

The following commands are used to install and verify GCC:

```
aayushdas@Ubuntu:~/lab2$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
aayushdas@Ubuntu:~/lab2$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
The following packages were automatically installed and are no longer required:
  linux-image-unsigned-7.0.0-30-generic  linux-tools-7.0.0-30
  linux-main-modules-zfs-7.0.0-30-generic  linux-tools-7.0.0-30-generic
  linux-modules-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 9
```

B. Four Stages of Compilation

The C compilation process consists of four main stages: preprocessing, compilation, assembly, and linking. Each stage transforms the source code into a different form until the final executable program is produced.

Stage	Command	Output
1. Preprocessing	<code>gcc -E hello.c -o hello.i</code>	<code>.i</code> file
2. Compilation	<code>gcc -S hello.i -o hello.s</code>	<code>.s</code> file
3. Assembly	<code>gcc -c hello.s -o hello.o</code>	<code>.o</code> file
4. Linking	<code>gcc hello.o -o hello</code>	Executable

C. COMPILATION STAGE EXAMPLES

1) Stage 1: Preprocessing

In the preprocessing stage, the source code is processed before compilation. Header files and macros are expanded, and the resulting preprocessed code is saved in a `.i` file.

```
aayushdas@Ubuntu:~/lab2$ gcc -E hello.c -o hello.i
aayushdas@Ubuntu:~/lab2$ ls
formatted  formatted.c  hello  hello.c  hello.i  input  input.c
aayushdas@Ubuntu:~/lab2$ cat hello.i | less
```

2) Stage 2: Compilation to Assembly

In this stage, the preprocessed C code is converted into assembly language instructions. The generated assembly code is saved in a file with the `.s` extension.

```
aayushdas@Ubuntu:~/lab2$ gcc -S hello.i -o hello.s
aayushdas@Ubuntu:~/lab2$ ls
formatted  formatted.c  hello  hello.c  hello.i  hello.s  input  input.c
aayushdas@Ubuntu:~/lab2$ cat hello.s
        .file     "hello.c"
        .text
        .section   .rodata
.LC0:
        .string   "Hello, World!"
        .text
        .globl    main
        .type     main, @function
```

The assembly file was generated successfully from the preprocessed file. The .s file contains the assembly instructions produced from the C program and can be used in the next assembly stage.

```
aayushdas@Ubuntu:~/lab2$ gcc -S hello.i -o hello.s
ls
cat hello.s
formatted formatted.c hello hello.c hello.i hello.s input input.c
        .file "hello.c"
        .text
        .section        .rodata
.LC0:
        .string "Hello, World!"
        .text
        .globl main
        .type main, @function
main:
```

3) Stage 3: Assembly to Object File

In this stage, the assembly code is converted into an object file containing machine-level code. The object file is saved with the .o extension and will be used during the final linking stage.

```
aayushdas@Ubuntu:~/lab2$ gcc -c hello.s -o hello.o
aayushdas@Ubuntu:~/lab2$ ls
formatted formatted.c hello hello.c hello.i hello.o hello.s input inp
aayushdas@Ubuntu:~/lab2$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
aayushdas@Ubuntu:~/lab2$ gcc hello.o -o hello
./hello
Hello, World!
```

The assembly code was successfully converted into an object file using GCC. The generated hello.o file was checked using the file command, confirming that the object file was created successfully.

4) Stage 4: Linking

In the linking stage, the object file is combined with the required libraries to create the final executable program. The executable can then be run directly from the Linux terminal.

```
aayushdas@Ubuntu:~/lab2$ gcc -c hello.s -o hello.o
ls
file hello.o
formatted formatted.c hello hello.c hello.i hello.o hello.s input in
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
aayushdas@Ubuntu:~/lab2$ gcc hello.o -o hello
aayushdas@Ubuntu:~/lab2$ ./hello
Hello, World!
aayushdas@Ubuntu:~/lab2$ echo $?
0
```

IV. RETURN VALUES AND EXIT STATUS

A. Understanding Return Values

The return value of the `main()` function provides an exit status to the operating system after a program finishes. A return value of 0 normally indicates successful execution, while a non-zero value represents another exit status.

```
aayushdas@Ubuntu:~/lab2$ nano return.c
aayushdas@Ubuntu:~/lab2$ gcc return.c -o return
aayushdas@Ubuntu:~/lab2$ ./return
Program executed successfully
aayushdas@Ubuntu:~/lab2$ echo $?
0
```

The program was successfully compiled and executed, displaying the expected message. The exit status was 0, which indicates that the program completed successfully.

```
aayushdas@Ubuntu:~/lab2$ nano return.c
aayushdas@Ubuntu:~/lab2$ gcc return.c -o return
aayushdas@Ubuntu:~/lab2$ ./return
Program executed successfully
aayushdas@Ubuntu:~/lab2$ echo $?
1
```

The program was modified to return 1 instead of 0 and was recompiled successfully. After execution, the `echo $?` command displayed 1, showing that the program returned a non-zero exit status.

V. KNOWLEDGE CHECK QUESTIONS

A. Question 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

The `#include` directive is used to include header files in a C program. Header files provide functions and definitions that can be used in the program. For example, `#include <stdio.h>` allows us to use functions such as `printf()` and `scanf()`.

Example:

```
#include <stdio.h>
```

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

The C compilation process consists of four stages: preprocessing, compilation, assembly, and linking. Preprocessing expands headers and macros and produces a `.i` file, while compilation converts the code into assembly language and produces a `.s` file. Assembly converts the assembly code into an object file with a `.o` extension, and linking creates the final executable program.

Stage	Description	Output
Preprocessing	Expands headers and macros	<code>.i</code>
Compilation	Generates assembly code	<code>.s</code>
Assembly	Creates object code	<code>.o</code>
Linking	Creates the final executable	Executable

C. Question 3

Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A return value of 0 from the main() function normally indicates that the program completed successfully. A non-zero return value indicates another exit status and is commonly used to indicate that an error or different condition occurred. The exit status can be checked in Linux using echo \$?.

D. Question 4

Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

The following program declares an integer variable and uses scanf() to receive a value from the user. It then uses printf() to display the entered value.

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}
```

E. Question 5

Q: What GCC command would you use to:

1. Only preprocess a file without compiling?

```
gcc -E hello.c -o hello.i
```

2. Generate assembly code but not assemble it?

```
gcc -S hello.i -o hello.s
```

3. Create an object file without linking?

```
gcc -c hello.s -o hello.o
```

The -E option performs only preprocessing, -S generates assembly code, and -c creates an object file without performing the linking stage. These commands were also used during the practical compilation stages of this lab.

F. Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

C is called a compiled language because its source code is converted into machine-level code by a compiler such as GCC before the program is executed. The compilation process produces an executable program that can be run by the operating system. In contrast, interpreted languages generally execute source code through an interpreter rather than producing a standalone executable in the same way.

VI. LAB TASKS COMPLETED

1. Task 1: Hello World Program

Created and compiled a basic C program that displayed “Hello, World!”. The program was executed successfully and its exit status was checked using echo \$?.

2. Task 2: Return Values

Modified a C program to return different values from the main() function. The exit status was checked after execution to observe the difference between 0 and 1.

3. Task 3: User Input

Created a C program that accepts an integer value from the user and displays the entered value. The program was compiled and executed successfully using GCC.

4. Task 4: Four Compilation Stages

Performed the four stages of C compilation separately using GCC. The intermediate .i, .s, and .o files were generated and the final executable was created through linking.

5. Task 5: Formatted Output

Created and executed a C program that accepts the user's name, age, and height. Different format specifiers were used to display the entered information in a structured format.

VII. CONCLUSION

This lab provided practical experience with basic C programming, GCC commands, and the Linux development environment. The tasks helped demonstrate how C source code passes through preprocessing, compilation, assembly, and linking before becoming an executable program. The lab also improved understanding of user input, formatted output, and program exit status.

VIII. GITHUB REPOSITORY

https://github.com/aayushbaniya69/240715_linux_reports.git